[Anna Rose] (0:05 - 1:59)

Welcome to Zero Knowledge, I'm your host, Anna Rose. In this podcast, we will be exploring the latest in zero knowledge research and the decentralised web, as well as new paradigms that promise to change the way we interact and transact online.

This week, Nico speaks with Alex Hicks, researcher at the Ethereum Foundation, for the 6th and final episode of our Lean Ethereum mini-series.

In this conversation, they explore the role of formal verification in the Lean Ethereum vision and how the ecosystem is working towards machine-checked guarantees across the ZK stack. From RISC-V, ZKVM, circuits, and constraint systems, all the way down to compilers and cryptographic primitives.

For these special episodes, we also have videos over on the YouTube channel. Links are in the show notes in case you want to check it out there, and be sure to subscribe to the channel while you're at it.

So this wraps up our Lean Ethereum mini-series. We would love to hear from you about what you thought about it. If you liked the video format, if you'd like to see us try something else, and especially if you have some ideas for future series we could do.

Before we kick off, three quick points.

First, zkSummit is coming back with its 14th edition. Applications to attend are now open. It will be on May 7th in Rome, so I hope to see you there.

Second, zkMesh+, our paid subscription tier, is rolling. We have released a few bonus clips over there, as well as some of our exclusive research reports. If you want to get access, please join the paid tier.

Third, if you're looking for other ways to support the show, we have donation accounts visible in the footer of our website. Support definitely goes a long way. So if you do decide to donate, thank you in advance.

That's all for now. Links are in the show notes.

Now here is Nico's interview with Alex Hicks.

[Nico Mohnblatt] (2:02 - 2:18)

Today, I'm here with Alex Hicks, Lead of Protocol SNARKification at the Ethereum Foundation. This is the 6th and final episode of our Lean Ethereum mini-series, and today we'll be talking about Lean for the first time, amongst other things.

Alex, welcome to the show.

[Alex Hicks] (2:18 - 2:19)

Thank you for having me.

[Nico Mohnblatt] (2:19 - 2:28)

Actually, this is really funny because I met you 5, 6 years ago before doing any cryptography, before even knowing about cryptography, because you were teaching at UCL.

[Alex Hicks] (2:28 - 2:29)

Long time ago now.

[Nico Mohnblatt] (2:29 - 2:38)

Yeah, yeah. But this is actually where I learned cryptography, was through the lectures and the tutorial classes that you were running. So it's a nice full circle moment.

[Alex Hicks] (2:38 - 2:41)

And you were clearly convinced to some extent that it was interesting stuff.

[Nico Mohnblatt] (2:42 - 2:57)

I mean, I hope so. Otherwise, I'm just really good at lying for the past 5, 6 years.

I was curious, actually, what was your story into, not necessarily cryptography, because clearly that dates a while back, but at least into the Ethereum Foundation and sort of this ZK ecosystem.

[Alex Hicks] (2:58 - 5:07)

It was quite a roundabout story in a lot of ways. So I think I got into... so originally I studied physics and then math, and I was going to do a math PhD. I had an offer to do a math PhD, I had accepted it.

And then I kind of felt like math, theoretical physics, academia was not looking that great in terms of job prospects and just vibes in general.

And I'd always had an interest in, I guess, kind of privacy and security and things like this. I think I used Tor before there was a Tor browser, when you had to use a client called like [?] Vidadia or something like this.

Not for any bad reasons. I was just getting DDoS'd in an online game and so I installed Tor.

And so I had an interest in that. I'd never really thought about doing computer science as a degree because I thought it was just writing code, which I thought was kind of potentially boring. I was not aware there was a thing as theoretical computer science, let alone, I guess, kind of cryptography or anything like this.

And so at that point, when I was kind of basically decided not to show up to do a PhD in math, I thought, well, maybe actually I know a bit more about all of this stuff now. I can see there's a lot of issues with privacy and whatnot.

Obviously, you had the Investigatory Powers Act in UK that was kind of showing up at the time. And so I thought, maybe this is something I should get into.

By basically pure luck, I did actually find people who wanted to take me on as a PhD student to do stuff related to this. And so I ended up pivoting into computer science and kind of information security more broadly.

I actually didn't pursue cryptography when I did this, because I read a lot of cryptography papers when I started my PhD, specifically around functional encryption.

And this was with kind of no background, so like, okay, the math kind of makes sense. I can kind of see this seems really cool. You could do arbitrary operations of encrypted data. Sounds great.

And then someone explained to me that IO was basically not a thing. And that's when cryptographers went efficient.

They meant runs in like polynomial time was horrible constants. And I was like, okay, this seems kind of maybe less practical than...

[Nico Mohnblatt] (5:07 - 5:07)

Lie.

[Alex Hicks] (5:07 - 8:40)

Not a lie. But like, I thought if I'm going from [?] spherical physics to computer science, like I want to do something practical. If I'm just going to do math, I could have just stayed there and done that.

And so my actual PhD was around actually kind of transparency, transparency-enhancing technologies, you could call it. So basically log-in mechanisms that have some cryptography that you would use to provide more transparency for systems that we use.

Blockchains obviously figure into this because they do basically function as big transparency logs in the same way, just certificate transparency might be another example. And at that time at UCL, there was obviously just a lot of people doing blockchain-related research.

A lot of them are actually still around, obviously. I was in an office with Mustafa who did Celestia, Ania who did NIM, Mary Maller who was the lead of the cryptography team for a long time at the EF, Sarah Azouvi who was at Protocol Labs, obviously George Danezis who did Chainspace and then is now chief scientist at Sui. Like there's a lot of people who are doing blockchain stuff.

So I naturally found myself doing blockchain-related research. And so I wrote my first paper related to Ethereum in 2017, which was about uncle blocks and the way they basically subsidised certain attacks on Ethereum. So that was a long time ago.

And so did all of this. So that was kind of neither really cryptography nor formal verification in a lot of ways.

There was a bit of cryptography, which was kind of understanding some of the cryptography, but obviously not writing papers about new cryptography.

But then around the time I was finishing my PhD, I thought I wanted to do something a bit more math-y again. And I'd done a bit of formal verification in between actually my master's and my PhD, because there was a guy in Cambridge called Lawrence Paulson, who was the creator of Isabelle, which is a proof assistant.

And he used to basically take on Maths MSc students for a couple of months after their master's to have them do mathematics in Isabelle and get kind of feedback about essentially the UX of doing maths in Isabelle.

And so around the time I was finishing my PhD, there was a lot going on with respect to Lean. It had started to kind of get some attention.

They had started to do basically interesting mathematics in Lean, which is something that's usually quite hard to do in proof assistants because you need to build up a lot of libraries to get to anything that's relatively modern and not just redo set theory or category theory for the tens of thousands of times.

And so I got into Lean basically around the time that I was finishing my PhD. I was doing a bit of... I was doing some Lean, I was getting a bit more into the ZK stuff because similarly I think it started to become a bit more practical and interesting in that sense.

And then I was also just procrastinating a lot with prediction markets, which I'm not... which I don't have time to do anymore.

And so I was doing that. I was interested in like the AI stuff related to Lean was maybe like very early on where it was kind of not as popular as it is now, but you had a sense that, okay, clearly if people are going to generate a lot of things with AI, then a proof assistant that checks your results is kind of nice and there's an obvious kind of synergy there.

And so, yeah, Justin posted this tweet, it was like, if you love AI and Ethereum, like send an email to this email address.

And so I sent an email that said, I'm kind of quite interested in the AI stuff, especially with respect to proof assistants and formal verification and what that could mean in terms of security and guarantees about the code and all of this.

It turns out that when Justin posted a tweet saying, if you love AI and Ethereum, send me an email, what he meant was we're looking for someone to lead a formal verification project.

[Nico Mohnblatt] (8:44 - 8:45)

So you were right on the money with your response.

[Alex Hicks] (8:46 - 10:48)

So I say, I think I probably lucked out to some extent of like, maybe not a lot of people mentioned formal verification in the email about their interest in AI and Ethereum.

And then, yeah, I had basically kind of... yeah, Justin and, I guess, Vitalik had kind of dreamed up this idea of obviously with all of the ZK, you had all of the L2s already, of course, you had zkEVMs, which were barely starting at the time in kind of 2024, with [?].

Clearly, things were trending in that direction, which was great, which was, I guess, had been the goal and the roadmap for a couple years, but it was becoming practical.

There were clearly a lot of issues with the reliability and the security of the software. Obviously, maybe issues with this is brand new cryptography that you're going from paper to implementation to production in not a lot of time. And you want to feel confident about what you're doing.

And so they thought, well, why don't we basically put in a lot of resources into formal verification? Why don't we actually have this kind of bets that AI is going to turn out to be useful for this?

And so, I basically found myself joining the EF in June 2024 to basically lead that effort with obviously, as Justin said on his episode, quite a big budget and a lot of resources. And I guess with the aim of trying to involve all of the teams who are actually doing formal verification in the ecosystem into this and making it a bit of a community effort.

And obviously we've seen even ZK Security now are doing stuff in Lean, I guess partially as a result of that.

And so that was kind of the idea was can we formally verify this stuff? Can we get people to buy in and to commit to doing this as a community effort where we stop what we're making more efficient use of all the different teams and the different skill sets, and can we actually like cover the whole stack of zkEVMs, which is quite a big stack and it's not obvious that you could actually cover everything.

But we've been making progress for the last kind of year and a half on this, which I guess is what we're here to talk about now.

[Nico Mohnblatt] (10:48 - 11:03)

Exactly. That's what we're here to talk about.

And also to kind of understand what is formal verification, because you've mentioned it a few times now. You've also talked about like theorem solvers or proof assistants, I think you said.

What are all these things? How do they differ?

[Alex Hicks] (11:03 - 12:48)

Yeah. Yeah. So this is always the important question is what are you actually doing?

So I think that there's a generic answer to formal verification, which is basically to produce a machine checkable proof of something, of some statement.

And the statement here could be like a mathematical theorem, or the statement is basically always a theorem. And then whether you define that as a mathematical theorem in the sense that it's from a math textbook or something about some software or something else, you have a statement and you have a proof of it.

There are different ways of obtaining these proofs. Some of these are proof assistants like Lean or Rocq or Isabelle or HOL4. And there's a whole kind of family of these, which have different logical foundations and different underlying type series.

And then you have things like SMT solvers, which are a bit different in a sense that SMT solvers are fully automated. You don't need to write proofs by hand in the way that you do in proof assistants. But they maybe have weaker guarantees in the sense that for performance reasons, they tend to be implemented in C++. And so you can have bugs in SMT solvers.

And you will typically... there was a thesis from someone at ETH Zurich a couple of years ago that found like 300 or 2000 bugs in some of the SMT solvers. And these are mostly like edge cases.

I'm not aware of anything that was verified with SMT solvers that turned out to be like wrong because of a soundness error. Typically because what you verify, you have some confidence that it's correct in the first place.

And so you have a lot of these trade offs of how good are the soundness guarantees that you have. And this varies even among proof assistants.

So different proof assistants, the way that they work is that you have a statement written in obviously a high level language that you can write and read as a human.

[Nico Mohnblatt] (12:48 - 12:50)

So this is like code, essentially?

[Alex Hicks] (12:50 - 14:28)

It is code. Yeah. So a theorem in Lean or proof assistant is actual code.

You have...I guess, some people will hear about the kind of Curry-Howards thing, which is that you have propositions or a type and then programmes or proof, which is basically an implementation of that type.

So your proofs are actual... your theorems and proofs are actual code in your proof assistant. And this basically gets compiled down to a kernel language. And then the kernel of a proof assistant is basically the core logic of that proof assistant against which everything is verified.

And so you basically want this to be as small as possible. And I think for some of, I think for Isabelle and HOL4, which are based on just higher order logic, which is obviously a much simpler logic, the kernels are actually very short, like hundreds of lines, not a lot at all. And everything gets checked against that.

So you have quite a high degree of assurance of there's no bugs in there, no soundness issues.

For things like Rocq and for things like Lean, which are based on kind of, you have dependent types, you have the kind of whole calculus of constructions, the kernels there are a bit more complicated. So maybe you're a bit less certain that like there aren't any soundness bugs.

I'm not aware of kind of important soundness bugs that have taken place, but there are like tricky ways that you can prove false with some tricks sometimes or something like this, right?

And actually, so your colleague Yoichi actually had a bug bounty to prove false in Rocq many years ago, which was due to I think Pollard's inconsistency, I guess is the reason which is that you could get it... you could do some tricks to get it to print false basically.

[Nico Mohnblatt] (14:28 - 14:33)

Even though you're trying to prove something that's not false. You're always trying to prove true, right?

[Alex Hicks] (14:33 - 14:33)

Yeah.

[Nico Mohnblatt] (14:34 - 14:34)

Okay.

[Alex Hicks] (14:35 - 15:06)

So if you prove false, so you have an inconsistency, right? And that's an issue.

So there is always whether you're doing an SMT solver, and it's like some C++ code base, which is kind of trusted. And for things like Z3str3, the code base is like maybe like hundreds of thousands of lines of code. So it is, there's quite a bit to trust.

And then even for proof assistants, you have obviously the actual kernel that you need to trust. And there are efforts to basically check the kernel with itself. So you have like Lean4Lean, and you have like MetaCoq.

[Nico Mohnblatt] (15:06 - 15:09)

Oh, so like a recursive proof of the kernel?

[Alex Hicks] (15:09 - 16:23)

With compilers, there's a basic test of like, can the compiler compile itself, right? And you can kind of think of it at the same time, it's the same thing like, can the kernel kind of check its own consistency?

And so there are efforts to do this, but they're not complete in any sense. They're quite far from complete in many ways. But obviously, empirically we're not aware of big soundness issues. But there's always like a small probability that there is a bug somewhere in the kernel.

And then you can do client diversity, right? So you have for Lean, you have different implementations of the Lean kernel. So there's one in Lean, there's one in C++, there's one in Rust, maybe there were more.

You could run your proofs against different implementations of the kernel, and maybe get higher assurance that way, as long as the logical foundations are not completely flawed, which is, not many people kind of dive deep enough into the logical foundations of all these proof assistants.

It's maybe like five a handful of people. Certainly not. At least I can't claim to have fully understood the Lean kernel, and not a lot of people can... for Lean, there's maybe like one or two people, Mario Carneiro, and maybe that's probably about it.

But it's still a reasonable...

[Nico Mohnblatt] (16:23 - 16:23)

That's a bit worrying.

[Alex Hicks] (16:23 - 16:48)

Well, yes and no.

I mean, it's not a big amount of code, relatively speaking, right? So it's still like, it is some trusted code, but it's some trusted code that everyone relies on.

And so if someone does notice that they've proved false for something that obviously shouldn't be, it's going to get flagged, because every Lean user depends on that same kernel.

And you do have obviously people who do commit a lot of time to look at it.

[Nico Mohnblatt] (16:49 - 16:53)

So in some way, everyone acts as a cannery for bugs in the kernel.

[Alex Hicks] (16:53 - 17:19)

Yeah. So we would obviously notice when we're formalising a bunch of coding theory to prove things about fraud, we kind of have a sense of like, we expect this to be true, and we expect other things to be false.

And if we accidentally prove something that we saw like this isn't right, we would notice it because we have some other form of ground truth, which is the papers and some sense that what we want to prove is correct, otherwise, we wouldn't be trying to prove it in the first place.

[Nico Mohnblatt] (17:20 - 17:34)

So actually, this takes me to a good question, which is, what do the theorems look like?

You said we formulate the theorem and we prove it. What do they look like? And why are people saying like, oh, I formally verified my Rust code base? What does that mean?

[Alex Hicks] (17:35 - 19:49)

Yeah, yeah. So the theorems look... I think quite close to theorems on paper, you have declaration, which is just like lemma or theorem, and then some theorem name. And so you could have like a basic example would be you have like a lemma add underscore com to say that like addition is commutative over the integers.

And then you would just declare two variables, A and B, declare them as type natural number. And then the statement itself would then be a proposition, which is that A plus B equals B plus A. And then you would have a proof by steps in the proof.

So it can be quite readable, especially some notation gets kind of ugly, but in the same way that some math notation gets kind of ugly and hard to read. And it also kind of looks like code because you're declaring variables, you're introducing variables, you can introduce assumptions or hypotheses, much in the same way you would on paper.

And so I think if you forced yourself on paper to really say like, these are the variables and this is their types on paper, you might use maybe sets rather than types as conventions go.

And then you would declare like, these are my assumptions, assumption one, assumption two, assumptions three. It looks kind of similar to this, with some syntax that depends on to proof assistant.

And then the proof is basically a series of steps, which again is, I think if you do like an introduction to proofs in like a math undergrad course, it looks familiar, right? When you're forced to do like, okay, I use this axiom for this and this axiom for that, and then whatever. It looks very similar to this.

And you can see if you're putting some Lean repository in VS code, you have like an info view and it tells you at every step in the proof, what the current proof goal is.

And so you essentially basically start with the statement that you're trying to prove, and then each step changes the goal until you hit something that is basically kind of obviously true and trivially true and that completes the proof.

And so one of the nice things about this is that this kind of reposition is a proof as code is that obviously a lot of software practises can kick in, which is one that obviously you need to maintain and debug and things like this. But also you can refactor and you can optimise, you can make the proof shorter or something like this.

You can make the proofs depend on different assumptions or use different lemmas.

[Nico Mohnblatt] (19:49 - 19:52)

Or more general, I guess, if you had for specific types and you realise...

[Alex Hicks] (19:52 - 19:59)

Exactly. And more general helps because more general gives you maybe better automation or lets you reuse more, more things.

[Nico Mohnblatt] (19:59 - 20:10)

Can you give us an example of where we do formal verification for code? Because at this point we were talking about like a basic math property, so addition, commutes. How does this translate to verifying code?

[Alex Hicks] (20:10 - 20:21)

Yeah. So when you want to verify codes, then your main responsibility, I think more so than doing the proof itself is to figure out what is the actual statement that you want to prove about your code.

[Nico Mohnblatt] (20:21 - 20:21)

Yeah.

[Alex Hicks] (20:22 - 20:59)

And this can be... sometimes this is not too bad. So sometimes you might have a specification for a function that's written in Lean and that you might've proved something about it in Lean or not.

And then you have some code in another language or maybe in Lean itself, but maybe it's like an optimised kind of version of the function, and you're going to bring that into Lean.

I'll say a bit about how you bring stuff into Lean in a second, and then you can assume [?] this of the form like the spec returns the same outputs on every input as the code or vice versa, or the spec implies the code.

And you can write some equivalent statement between what you're trying to prove against your spec.

[Nico Mohnblatt] (21:00 - 21:02)

So always comparing code to a spec?

[Alex Hicks] (21:03 - 22:23)

Well, that's one way of doing it because then you're reduce things to like a nice Boolean, right? So it's like, if spec, then code. So something along these lines.

And so this is kind of nice. And this is, you can kind of see how that looks like as a theorem, like if A, then B. So it's not too different.

The trick there is obviously you have to get the spec right. If you get the spec wrong, you're in no man's land.

So the specing part is very important. More important than writing the proof in a lot of ways. And then there are other things that you might want to prove about your code.

So you might... for cryptography or for some cryptography, you want to prove that the code runs in constant time, or you might want to prove panic freedom about it or properties of these things, which are much farther removed from maybe mathematical properties of the functions that you're interested in implementing in your code and much closer to the behavior of the code, generally speaking.

And there, I think it's kind of interesting because then you have another element of trust that comes in, which is that you want usually to extract code that's not written in your proof assistant into your proof assistant.

And so you have to translate that code. And because I've seen most languages that people use, whether that's Rust or C++, Python, Go, whatever, most of these don't have formal semantics. For C, for example, there is formal semantics.

This is for C99 specifically, there is formal semantics.

[Nico Mohnblatt] (22:23 - 22:24)

What do you mean by formal semantics?

[Alex Hicks] (22:25 - 23:43)

Well-defined behavior of every part of the language. So the semantics, maybe semantics is kind of different depending on what you're doing. It's like not a word that gets used a lot.

But the semantics are kind of the meanings of the code. So if you have like a function and you're thinking like, what are the semantics of this function, the semantics should be like, what does it do, basically.

And then how you implement it is basically you want to check that the implementation produces behavior that matches those semantics.

Otherwise you have a bug or you have undefined behavior and then you can't reason about what it's going to do.

So when you're extracting code or translating code from any of these languages that people actually use for software development into Lean, because these don't have formal semantics that you can use as a reference.

You can't really do proofs of this Rust code translated to Lean is actually the same as this Rust code with some kind of equivalence relation. You're not able to do this because you don't have a reference for what are the formal semantics for the Rust code.

So that's kind of trusted. And so you can do a lot of testing, obviously. You can get smart people in the room who understand both languages very well and get things like if you have both of these things, their behavior is going to be the same.

[Nico Mohnblatt] (23:43 - 23:45)

Or you can write the spec, I guess.

[Alex Hicks] (23:45 - 25:04)

You have to kind of... well, you basically what you end up doing it, you kind of do semantics for like a subset of Rust.

And for example if you're doing purely functional Rust, that's much easier to deal with and to reason about than if you're doing very imperative Rust, it might have a lot of side effects. Like if you have mutable borrows, if you have code that returns a mutable borrow, that's kind of hard to like reason about. And it's kind of hard to map this, like, how do you do this in Lean in a way that kind of makes sense.

And so all of the tools that we have to translate from some language to Lean, and we have some for Rust, there's Aeneas, there's Hax, there are others which translate also Rust like some SMT, sort of a representation. You'll see the word "high assurance" used sometimes instead of "formal verification", which is a much better term because it kind of translates a bit more there's a level of assurance which is high based on testing and auditing and people using it, but there's no proof of that translation, right?

And so what you have instead is you have a proof in Lean that the way that you've modelled your Rust code in Lean, you can show something about your Rust code.

And so you're trusting the model of that Rust code and then you're trusting the Rust compiler and all of this that goes on after.

Now so that's quite a big... trusting the Rust compiler and LLVM and all of this is quite a big trusted computing base.

[Nico Mohnblatt] (25:04 - 25:06)

Yeah, but it's a lot of us relying on it, right?

[Alex Hicks] (25:06 - 25:29)

And there are bugs in LLVM, right? I mean, there was one very funny bug, which was someone had an instance for some reason, once all of their code ended in an LLVM IR, they had a function which was like f of x equals x minus 4, and it had actually just been rewritten to like f of x equals 4.

And so you can kind of imagine if you have this, that might happen to your constraints or something like this.

[Nico Mohnblatt] (25:29 - 25:32)

Yeah, yeah, yeah. Sure, everything passes completely.

[Alex Hicks] (25:32 - 26:00)

Yeah. You're gonna have some surprise.

And so there's this kind of trusted code base and understanding the boundaries of where your formal verification begins and ends is really important. In the same way that obviously if you have code in libraries, you might have a library that has like 10s of 1000s of lines, and you might have some core functions, which are formally verified, and then the rest is not.

And so you can still have bugs and ideally, you want people to document things well.

[Nico Mohnblatt] (26:00 - 26:10)

Typically, if you have a code base with 20,000 lines of code, let's say, but a lot of those are tests, you don't need to formally verify the tests, right? You can just stick to the core functionality.

[Alex Hicks] (26:10 - 26:11)

Yeah, yeah.

[Nico Mohnblatt] (26:11 - 26:11)

That makes sense.

[Alex Hicks] (26:12 - 27:13)

But then you can always keep going... you can always chip away and go further, right? So you can always think about, okay, do you want to formally verify the compiler? And you have... there is a formally verified C compiler called CompCert, which was this insane borderline in human effort that took place in like 2008 around these times.

However, many postdocs and PhDs spent their their years on this. But there is a formally verified compiler for C. And again, there's a boundary, the front end to that is not formally verified, it's just once you're in the compiler, and basically, every pass in the compiler is formally verified, you have a proof for every pass, every rewrite, every optimisation pass, all of that is basically proven against a C99 specification.

So you trust the C99 spec, but obviously, the spec is much smaller than the compiler code base. And then you trust a OCaml compiler, which compiles your OCaml code in which CompCert is written.

But then obviously, maybe a OCaml as a compiler is like more trustworthy than C.

[Nico Mohnblatt] (27:13 - 27:21)

Or smaller or easier to look at.

So this notion of boundary is actually helpful, right? Because we keep pushing further, like, what's the latest thing that I trusted?

[Alex Hicks] (27:21 - 28:41)

Yeah. You keep pushing further every time, and you want to keep track. So it's important to kind of really document and keep track of like, within your entire stack, what are you assuming is secure? What are your compiler toolchains? What optimisation...?

Because with LLVM, you have like O1, O2, or different levels of optimisation that you can apply. And it might be that you might trust O1, you might trust O2 or O3. You might trust a specific version of LLVM or something, because there are instances of some code that used to be constant time written in C or something, and they've put a lot of it into making constant time, then someone updates Clang, and they added like a smart optimisation pass that makes your code not constant time.

So these are things that come up and one of the challenges is that you want to keep track of your boundaries, and you want to keep track of how your boundaries change or stay the same over time. Because obviously, if you're doing... you want to verify some Rust code in Lean, Rust is still changing, Rust compiler is still changing, and Lean is still changing.

And so you do want to make sure that you have pretty solid, basically engineering practises to maintain all the whatever level assurance you claim to have, because the whole stack is always moving. And a lot of this is actually kind of dumb in a lot of ways.

[Nico Mohnblatt] (28:41 - 28:42)

Do you say dumb or done?

[Alex Hicks] (28:42 - 28:43)

Dumb.

[Nico Mohnblatt] (28:43 - 28:45)

No, okay. Just checking.

[Alex Hicks] (28:45 - 30:08)

No, far from done, unfortunately.

But it's an interesting thing, which is also that like, why do we have all of these boundaries that are there, that we constantly have to keep track of, which are all moving in different directions independently maintained by different people, which makes this inherently hard as a result.

The nice thing would be, well, why don't we just write code that is correct by construction or something like this?

And I think this is something that for a long time is, you'll see people mentioned about you'll see it in academic papers, you'll see prototypes like [?].

If you want to do like a small, purely functional language, and you want to build a compiler just for that, with the intent of making it easy for me to verify, but it's kind of doable. You'll see people have done this as their PhD, or just one paper or something.

What's harder is getting something that's as fast as Rust or C++, and heavily optimised for different hardware architectures and different backends.

And implementing reasonably complicated algorithms where you can have bugs in the code, you can have bugs in the spec.

Plonky3 had a bug, which was in the spec. It wasn't doing the final degree check for the FRI verifier.

And so, but it's a noob.

So this is again, like in a large enough library, you have empirical studies of like, if you're given like a thousand lines of codes, regardless of the language, you have somewhere between one and 50 bugs, depending on the quality of the code. Like one bug for a thousand lines of codes is considered state of the art, you can put it in like a fighter jet or something like this.

[Nico Mohnblatt] (30:08 - 30:08)

Wow.

[Alex Hicks] (30:09 - 30:48)

And so if you have like tens of thousands of lines, like statistically you know there are bugs basically.

And the same applies for specs. If you have tens of thousands of lines of specs, and it's still someone writing them, like they're easier to read than optimised codes, of course. And it's like, you might have like a reference on paper, but it's still tens of thousands of lines. So you can have human error and things like this.

And you have you know... in the context of zkVMs, for example, one of the nice things here about RISC-V is that the RISC-V spec is quite short.

If you look at like some of the ARM specs, they're like above a hundred thousand lines of specs, which is like, you can have issues, right? And they're evolving, right?

[Nico Mohnblatt] (30:48 - 31:04)

Based on the previous math, easily.

Actually, you're bringing us perfectly into what I want to talk about, which is formal verification of the Lean Ethereum stack.

What is being formally verified? What falls within the boundary? What is out of it? Tell us a bit more.

[Alex Hicks] (31:05 - 31:42)

Yeah. So it's a good kind of segue into this, because obviously, it's one thing where we do need to track what the boundaries are and what we're actually doing and what we're trusting.

So a lot of the focus right now, I guess, is whilst obviously LeanVM and all of this, which I guess would have been on previous episodes that haven't been released yet, but will be soon, I'm sure, at least at the time that we're recording, then LeanVM is not in production yet.

So right now, for example, we're not formally verifying anything about LeanVM specifically. So we're very focused on these RISC-V zkVMs. So kind of SP1, OpenVM, Airbender, ZisK, Jolt.

[Nico Mohnblatt] (31:42 - 31:44)

So this is all like the execution layer stuff.

[Alex Hicks] (31:44 - 32:38)

Yeah, the execution layer. The stuff that's like on these proofs, basically. And here we're trying to cover different parts of the stack basically in parallel.

So if you go from the ground up on the side of the actual like zkVM, especially the like RISC-V components of the VM. So here, what we've done is we've extracted the official specification for RISV-V, which is written in a language called SAIL.

We basically now have a Lean backend for SAIL. So SAIL can emit Lean code.

And so we have the entire RISV-V spec and actually other specs that are in SAIL, which includes I think some ARM and some x86 and things like this. But we have the entire RISV-V spec that can be extracted into Lean.

So that includes every extension. So whatever extensions people want to use in the future, we have specs. And therefore, with these specs, what we can do is we can then actually...

[Nico Mohnblatt] (32:38 - 32:40)

Is that backend part of the boundary? Or is this something we're trusting?

[Alex Hicks] (32:40 - 32:42)

That backend is part of the boundary. Yes.

[Nico Mohnblatt] (32:42 - 32:42)

Okay, nice.

[Alex Hicks] (32:42 - 34:23)

Because again, like the spec for RISC-V is kind of... There is some basic checking. There's some like, you can apply some SMT solvers in SAIL or in Lean, you can write like some lemmas of like this function that's going to take a bit vector of like this length, is going to like just take a bit vector of this length and nothing else.

And you can do basic [?]sanity checks like this. But obviously the spec is whatever the RISC-V Foundation decides it is.

And then the implementation of that spec is whatever the people working on the spec in Cambridge and SAIL have implemented it as. And then the backend for that is again, like a mapping from basically SAIL to Lean.

Now, luckily, I think the two languages are not too dissimilar in like the underlying logic that it's... I feel pretty happy and confident with it. And then it passes... there's a test suite and you can pass the official test suite and things like this

But that part is trusted. But there's an interesting discussion, which is like, is it better to trust this backend, or would it be better to trust like a handwritten RISC-V spec?

And so actually what we have is we have a handwritten RISC-V spec, which is nicer because it's bit [?] blastable, which kind of helps with verification, and we've shown it's equivalent to the extracted SAIL spec.

And so we have some sanity check here. We have an extracted spec and a handwritten one and they're both equivalent.

So those proofs cover the pure instructions, the instructions that have side effects like obviously load and store and things like this.

So with this spec, which we're trusting to some extent, then obviously what we're able to do is then we're able to get the constraints for the zkVM RISC-V sockets and we're able to verify these constraints against the RISC-V spec.

[Nico Mohnblatt] (34:23 - 34:27)

I see. And how are the constraints mapped into Lean?

[Alex Hicks] (34:27 - 34:37)

The constraints are mapped into Lean currently, mainly you pass them off of the AST from the source language in which they're written. And then you basically print that back to Lean.

[Nico Mohnblatt] (34:37 - 34:39)

And again, part of the boundary or not part of the boundary?

[Alex Hicks] (34:39 - 35:07)

Part of the boundary. Yeah, exactly. Because obviously if you take someone might have written their circuit with Plonky3 or some other Rust DSL.

And obviously, again, you hit this issue if you don't have formal semantics for Rust, you suddenly no formal semantics for like those DSLs. And so you just read off values from the AST.

Now, luckily, this doesn't feel horrific, right? One, because obviously the AST is still reasonably high level, not much has been done to these constraints.

[Nico Mohnblatt] (35:07 - 35:12)

By the way, the AST is kind of like, almost like a binary of what the constraints are?

[Alex Hicks] (35:13 - 36:41)

So the AST is kind of once you take your high level code, and then you try to represent basically as a tree of operations, that's the AST. And so if you have like a loop or something you might have all the different values as a loop.

And so that gets extracted. And so it looks kind of ugly, actually, that's the main thing.

The main thing is it's hard to... the constraints are kind of ugly. And I'll say in a second how we can do this better.

We've got this boundary. So we extract the constraints into Lean. And then so we have a bunch of extracted constraints into Lean, which we're trusting match those that are generated by the Rust code, when you do actually run it through your normal compiler and you deploy it in production.

And you can kind of check this if you want, which is that you could actually run your regular things through the Rust compiler as usual, and just check that the constraints are the same.

So this you can kind of test actually fairly well. And because the constraints are not something that has... they don't have like an insane amount of churn, it feels pretty reliable that you're not... no one is doing like a commit to the constraints every day, we need to test it, and so on.

So that that feels like... it feels kind of okay.

And then the proof that you do is you have proofs that are basically of the form that the spec implies the constraints and the constraints imply the spec.

And so you show that obviously, one, that the constraints are satisfiable. So you are going to be able to generate a witness. And two, obviously the constraints are sound in a sense that obviously they don't allow... they can't be satisfied by multiple different witnesses.

[Nico Mohnblatt] (36:41 - 36:48)

Do we also need to check that this witness actually satisfies the Lean spec or like... Sorry, not the Lean spec, the RISC-V spec.

[Alex Hicks] (36:48 - 40:15)

The problem is with this is that the zkVM teams have been hard at work optimising their witness generation. So there's not a performance bottleneck. And so whilst you could in principle have circuit DSLs where the witness generation and the constraints are coupled, and you'll see this in Circom, for example, or even actually with Plonky3, right?

But in practise, people don't do this in production, they have completely decoupled witness generation from the constraints.

And so when we're extracting the constraints, we're not extracting the witness generation logic. And so we're only showing completeness in the sense that like, they are satisfiable, we're not showing to the witness generation that the zkVM is actually run, produces a witness that always satisfies these constraints.

We are trying to resolve this. So you have all these boundaries, basically. So you have these proofs in RISC-V, and you can find detailed reports.

There was one done for the OpenVM RISC-V constraints. The work to verify this was finished earlier this month. There's a very detailed report in a repository there, which maybe [?] or something, we can put it there. It goes into detail, and you can see that the proofs.

And so this will feel solid. But again, you have boundaries, right?

And so there's an interesting question, like, well, how can we improve on this? Because one goal with all of this formation stuff is not just to do it once, it's also to try to make it actually kind of easy to do, and economically sustainable and not cost like a ton of money.

And so there are two things. One is you could just write your constraints in Lean. And obviously, you have DSLs like Clean or zkLean, which are doing this, right? And this actually turns out to work reasonably well.

Lean is a good functional programming language, so you can write these kind of things. And then the proofs, if you've written your constraints in Lean, are much, much nicer by a factor of 10 to 100x, I would say.

I mean, one of the first examples that was in Clean was obviously like a Keccak precompile. And then the proofs there, soundness and completeness of each permutation, like 10 lines each, and then a proof for the composed full round is like 70 lines for soundness, 70 lines for completeness. So that's actually pretty good, right?

And that's a reasonably optimised Keccak precompile. I don't know if it's as optimised as the Plonky3 one, whatever. But that's really good compared to the Plonky3 Keccak, where actually extracting those constraints into Lean doesn't work, because if you put the constraints in a file, it's about 1.4 gigabytes. And if you try to open a 1.4 gigabyte file in VS code, it crashes.

And so we did actually verify the Plonky3 Keccak in Rocq. And the way that this was done by Formal Land was that they actually basically extracted the Rust code from Plonky3 to Rocq. And then they basically checked that that was generating the exact same constraints as the Rust code would on its own.

And then that obviously let them do basically proofs about like the functions and the high level description of the programme itself, in the same way that you might do in Lean with a Lean DSL, rather than trying to extract all of the constraints in one and do this.

So there's a lot you can do for proof ergonomics, and for the proofs to be easier to do. And obviously it's easier to maintain, because proof... this is another kind of maybe dark secret is proofs break, not in the sense that they're not sound, and like the soundness is compromised, but in the sense that you have like a bump to the version of Lean or something.

And then there's a dependency in mathslib, where someone tweaks the definition of something in mathslib, and then it breaks your proof, and then you just fix the proof.

It's not a big deal.

[Nico Mohnblatt] (40:15 - 40:31)

Actually, this is a good point. Is there like an ecosystem of interdependent proofs? Because obviously, if I'm using basic math theorems, I'm going to use it from someone else, right? I'm not going to write it from scratch. So is there like a canonical math library that everyone uses?

[Alex Hicks] (40:31 - 40:52)

Yes. So there's this big library called mathslib, which has... at least like two years ago, it was well over a million lines of code. And codes there being just like mathematical statements and proofs. So by now, it's probably like a couple million.

It's fair to say that it roughly covers all of undergrad math that you would ever need. So it's fairly extensive.

[Nico Mohnblatt] (40:52 - 40:57)

And so if this changes, then every proof you've written needs to be changed, or you need to somehow bump it up?

[Alex Hicks] (40:58 - 43:35)

Not every proof, but every now and then you get... I mean, sometimes someone changes like a file pass. So at some point a couple months ago, they moved a bunch of theorems about matrices from one file pass to another. And so that broke a bunch of our proofs.

But it's just you change the import and it fixes itself. And then you have other things like someone might change a definition, things like this. And then you just have to fix your proof.

And so, it's not that the proofs are not sound, it's just that there's churn in the libraries and in dependencies and you kind of address them.

The nice thing is obviously when your proofs break is very precise. You have like within a proof, you might have a hundred lines or whatever. One line goes red and you know exactly where it breaks and you fix it.

So it's kind of okay. But it's an interesting thing that you have to know that because it's code, you have to maintain it, right?

You can leave a repository tanked for a version of Lean and not bump it to newer versions of Leans and then it would break, right? If the code is static and the dependencies are static, it's fine.

But it is a thing. And so if you want to kind of keep maintaining proofs of software, that is whether that's constraints or some of the kind of software where there is churn in the software and you want to also maybe keep up with new versions of Lean because you get better automation and other like maybe helpful lemmas get added to mathslib to like solve your problem or just other features get added to Lean which solve your problem, which actually does happen quite a lot, luckily for us.

Then you know, there is a bit of churn and maintenance and it might take like 30 minutes a month or an hour a month where you fix like a couple of proofs.

It's a good way to understand proofs that you haven't written if you're responsible for maintaining them, I guess. But you kind of have some churn, but it's okay.

It's not the worst and it's not a soundness issue in any sense. The biggest soundness issues tend to be if you've like verified some code, add a specific commit hash in the source repository of the code, and then obviously if you change the code and you haven't done a new proof or something, and that's something that I think is maybe more concerning because a kind of a practical limitation, which is kind of annoying is you don't really have something in CI that extracts your constraints back to Lean and then runs all the proofs again and verifies that the proofs are still fine.

One, because obviously running the extraction and then building the Lean file in CI can be kind of long and might not work out in terms of just lengths and resource allocation in CI. And two, because it depends on how robust the proofs are to changes.

So if you have a proof, which is extremely manual, where the proof is extremely kind of tailored to that specific constraint, that if you change it a bit, it might break.

It won't break in like drastic ways. Like the constraints will still look the same.

[Nico Mohnblatt] (43:35 - 43:38)

But you still have to go into the proof and change it.

[Alex Hicks] (43:38 - 44:03)

And you have to go in and change it. And it's kind of annoying.

But you have proofs which are increasingly automated. So you have decision procedures for statements about polynomials of a finite field, right? And then if you have, if the proof goes through some automation, then actually the proof might actually hold up to changes in the constraints.

And so increasingly we're seeing more robustness in the proofs and we're obviously working to have more robustness in the proofs so that they don't break as much.

[Nico Mohnblatt] (44:04 - 44:23)

Actually, automation is a good point. And one of the last things I wanted to talk about.

You mentioned earlier an inhuman effort, which made me think, ah yes, inhuman efforts. Actually, we're at a point where inhuman efforts are probably really good. How is AI and LLMs participating in all this proof systems kind of stuff?

[Alex Hicks] (44:24 - 48:38)

Yeah, well, so as of recently, they're participating a lot. We're probably fairly close to it. I mean, there's two things.

There's one thing, which is that, so AI, I think for people who are doing things around AI and formal verification or people in formal verification who are looking at AI, there was always an obvious synergy, which is AI is basically good at producing large amounts of outputs. And proof assistants or SMT solvers or any kind of thing that checks that is machine, some machine-checked process are very good at handling large amounts of outputs and telling you if they satisfy some kind of property.

And so, there was always a sense that, well, if you are going to have these otherwise unreliable models that can output things that look like plausible proofs of statements or things that codes that you might trust or not want to trust, why don't you just put that into a proof assistant, and then a proof assistant will tell you if you can trust it or not.

And if you don't trust this, you just rerun this thing until you get some output that satisfies it. And that seems like a nice thing.

It took a bit of time to materialise, but it's kind of there for some things, right? So for proofs in particular, it is basically there.

So the very first period was Justin, he mentioned it in January. I launched a proof run with using like a specialised model developed by Logical Intelligence, which is specialised kind of mathematics and software verification and things like this. And in particular, basically can be stable for like multiple hours.

And so that took eight hours and it proves Polishchuk-Spielman, the form that appears in the proximity gaps paper in eight hours at a cost of $200 in compute, which is really nice because I think by comparison, if you gave that to a human expert, which is familiar with the coding theory as is defined in ArkLib and you know, the proof itself, they would take at least a couple hours, like they would take a couple hours, but they would cost certainly more than $200 because they might charge more like $200 an hour and take a couple hours, if not maybe a day or two or something.

Maybe less, maybe more like there's different levels of expertise and competence, but there was a bottleneck, which is kind of human time to do all of these things.

Even if a lot of these things felt very doable and we had high confidence were doable, there was a bottleneck, which is just a lot to do.

So o get kind of formally verified specs for FRI, that is like an executable FRI protocol where we showed it you have a statement in the paper, which is this construction has some soundness property.

And we want that to be in Lean in the form that when we say this construction satisfies soundness, that construction is an actual executable piece of code.

It's not like an abstract function. But that required us obviously to one, write the specs, which took quite a bit of time. We had to do a lot of abstractions for like Oracle frameworks and things like this, which [?] was working on.

But also we had to do actually coding theory from scratch because mathslib covered basically linear algebra and everything you need, but it didn't have any coding theory beyond like hamming distances. And so that's a lot of work because it's just a lot of material.

And so we were able to get all of the statements written down and we had some of them actually definitely had some issues. Some of them were stative, but like two general algebraic structures or typos and things like this, as you would expect.

But now the nice thing is obviously we made a conscious bet, like basically not prove everything as we went along so that we could go faster and like getting to getting a FRI spec.

And on the assumption that proofs would just get easier, either with better automation in Lean or with AI and things like this. And then it's kind of worked out because now actually we have all these statements. We actually have a spec for FRI.

And then we have AIs, which are able to do hard theorems like Polishchuk-Spielman, which is really cool. But also for a lot of simpler theorems, we've had a lot of first time contributors who just open PRs and who have proof of some statement or a couple of statements. A lot of these are increasingly AI assisted.

We even have like... we have some AI in like the way that we manage the library. I have a script that scans for missing proofs in the library and then opens GitHub issues and documents them with the proof from the reference paper that that thing is taken off.

And so we've had a lot of people who've kind of opened PR to say, well, I followed the offline proof in like this issue and I use like Claude or Codex or whatever, and like here's the proof.

[Nico Mohnblatt] (48:39 - 48:40)

And if it verifies, then we're happy.

[Alex Hicks] (48:41 - 48:47)

And if they haven't changed the statements, I just need to check that it builds and I just merge it. So that's kind of nice.

[Nico Mohnblatt] (48:47 - 48:56)

Definitely. From our side, we've seen Claude getting much, much better at doing this. And my colleagues have been having a lot of fun proving things in Lean from papers.

[Alex Hicks] (48:56 - 50:07)

Yeah. And so that's really cool because it really takes away one part of the job. The part of writing specs is still... it's not solved or you're not letting an AI do this automatically.

It's still the most important. It's where you can get it wrong, especially for things where the specs can feel a bit arbitrary where... for things like cryptography that has provable security, if you write down a protocol and then you prove something about it, you at least have a sense of like, if you did actually run some... if you run this, or if you verify something against it maybe, I don't know, it happens with like... it's not actually FRI, but it's still a secure protocol.

So maybe you've discovered something new, which is fine. Whereas obviously if you're doing like a hash function or you're doing like Fiat-Shamir and you have like a spec, but you can't really prove that it's like good in any sense, then obviously you want to have some oversight on checking that this actually is like what you expect it to be.

And so there's still a lot to work on, like doing specs and writing specs and getting better abstractions and still actually like library management is something that obviously takes time and effort to do correctly. But the proof part, it now feels kind of solved, which is extremely exciting.

[Nico Mohnblatt] (50:07 - 50:09)

So it sounds like you made the right bet.

[Alex Hicks] (50:09 - 53:48)

It sounds like we made the right bet. And it's been one of the nice things as well is because the proof part, which previously would obviously be quite tedious, because sometimes it feels kind of pointless. Like why are you proving in a proof assistant, something that's been proven on paper like years ago that everyone feels fine. There's been whatever.

And it can be a bit of a bottleneck as well in trying to get people to buy into like doing things in a proof assistant.

So we've been thinking like, how do we get cryptographers to just implement their new cryptography directly in Lean so that we don't need to like go through the whole deal of having to formalise it and translate it and everything.

And I would say that like a year, a year and a half ago, two years ago, it'd be very hard to kind of convince anyone to pick up Lean for that purpose.

But now because we're able to say like if you just write the statements down, which is a much easier thing to do than obviously doing the proofs... we had a very fun example of this recently, which is that we have a spec for a single round of sumcheck in ArkLib.

And we're working on making it executable. We have like a computable polynomial library, which makes it executable. But it's not like the full protocol as you would run it.

And we had Andrew Zitek, who is a research engineer in Alexander Kessler's group at EPFL. He's worked on sumchecks, he's implemented it in Rust and things like this, started kind of looking at Lean a bit. And got into the idea of trying to write like an executable sumcheck protocol in Lean as a general kind of functional programme.

And I had a chat with him in December. I think he'd obviously been looking at it already. He talked to [?] Kuang before that, but it wasn't done at that point.

And in January, I get a message like, oh, okay, I have it here. Here's the repository. There's a sumcheck protocol. You can run it, it's computable, it uses our computable polynomial library. There's a bunch of proofs that are there for soundness.

There's a few lemmas which are missing. And I see this email on Friday evening, and I just do a couple of proof runs. I go to bed, I wake up Saturday morning, and I'm like, hey, here's PRs with the proofs that you're missing. And then it's done, right?

And that's kind of nice, because it means that if this is your first experience with Lean, and you might otherwise think like, oh, I've written the programme, and I've implemented it, I feel this is secure. I just don't know how to prove this, because it's just kind of annoying to prove things in a proof assistant compared to doing it on paper. And now actually, you just click a few buttons on a web portal, and you get a proof. That feels really nice. That's like a big boost.

And so I think the things for us kind of moving forward are going to be much less kind of trying to deal with all of these boundaries that we mentioned, and trying to actually kind of embed Lean and other tools [?] like directly into the way that people do research and software development.

And that includes we kind of like, can we get people to kind of write down the spec for their protocol in ArkLib, given that we have all the prerequisites. Like if anything is coding theory based, like everything is there now, you would be able... if you do like an improvement on WHIR you should be able to state it in ArkLib.

And suddenly, if you can automate the proofs, you should be able to state it and get proofs much sooner than anyone can peer-review it, I think. Then that'd be like a goal.

Same with like, if someone does like the proximity gap stuff, I feel like we should be able to do it in Lean, verify it in Lean before it's peer-reviewed. I think that would be like a nice thing.

And same for circuits, right? If you can do like a nice Lean DSL for circuits, just write your constraints in Lean, write your widgets generation in Lean potentially, and then you can just prove things.

And it's just much easier to prove things. Lean is a good functional language. If you like functional languages, you can kind of use it.

[Nico Mohnblatt] (53:48 - 53:49)

That might be a big if.

[Alex Hicks] (53:49 - 54:08)

Big if, yeah. But it's good enough for some people, right?

And for a critical software, it's okay to not have everyone write this code. And it's the way that it's done, right? There's a huge centralisation issue with like everyone uses Plonky3. So if there's bugs in Plonky3, like multiple zkVMs can be affected, right?

And so you might as well, if you're going to do that, you might as well do it.

[Nico Mohnblatt] (54:08 - 54:09)

Put in the effort. Yeah, absolutely.

[Alex Hicks] (54:09 - 56:30)

And then obviously the next step after that is, because we talked a bit about the boundaries, is always the compilers, because these compilers are like huge things. And LLVM is like this huge kind of massive IR that is hard to do much about because it is just huge.

And so the next boundary for us is really kind of like, well, can we just do things in Lean natively? And we save ourselves of extracting a lot of the stuff into Lean. And then can we write actually compilers that are kind of limited and easy to formally verify? And which actually outputs hopefully actually more performant codes than the Rust compiler might.

So that's the next... that I think is the kind of next thing, because you have... for cryptography specifically, I'm not talking about programmes in general, for cryptography specifically, you don't need your compiler to do that much. You're operating on... basically, you're doing finite field arithmetic, you're doing operations over polynomials. So you need optimisation passes related to this.

You need Montgomery reduction or Barrett reduction all of these things. But you don't need everything that you might find in LLVM IR.

You have a lot of optimisations which operate at a reasonably high level, because they're probably like mathematical in nature, rather than learnable software optimisations. And you have things like mutating in place inside an array, which you might not do that much of.

So the compilers that you need for like a proof system, for a verifier, let's say for a verifier, because then you can run it on CPU and not rely on GPUs, you don't need that much, right?

And you have these compiler frameworks like MLIR, which are based on this idea of having these small dialects that have a couple types, a couple operations, and you just have standardised data structures and a standardised way of lowering from one dialect to another, all the way down to assembly or whatever hardware backend you want.

So you have these compilers that you can write, which might use 4 or 5 MLIR dialects. And that's it, at least backend... and then you have like hardware backends, which would be unverified.

And we now have MLIR data structures, which are re-implemented in Lean and which are asymptotically fast. And so you can write a compiler in Lean with data structures, which are proven memory safety and all of that is proven.

So the next step for us is basically to kind of compile from ArkLib specs to whatever hardware backend via a compiler that we're going to write in Lean, and which we're hoping is going to be faster than Rust.

[Nico Mohnblatt] (56:30 - 56:38)

So we're bypassing research on paper, bypassing coding into Rust, everything goes into Lean directly and we have fast executables, and we're happy.

[Alex Hicks] (56:38 - 57:18)

Yeah, because then you kind of go full circle to... the idea of LeanVM is to kind of minimise a lot of these assumptions. And to have actually like, you have a tiny ISA, it's very easy to formalise.

You have specs and you have like a compiler in between, which doesn't need to do much, because you have like 5 instructions, you know. And so you're kind of compiling down to that from ArkLib or some kind of DSL.

The scope is not too bad. It is actually possible to just like formalise this. And then you get to the thing of like, how do we minimise boundaries? And it's like having a kind of Lean stack that you can actually provide assurance for as a whole.

That's the end goal, I think.

[Nico Mohnblatt] (57:18 - 57:23)

Well, looking forward to it.

Alex, I want to thank you so much for coming on the show.

[Alex Hicks] (57:23 - 57:23)

Thank you.

[Nico Mohnblatt] (57:24 - 57:34)

And I'd like to also thank the podcast team, Anna, Rachel, Henrik, Tanya and Hector.

To those of you catching us on video, thanks for watching.

And to our listeners, thanks for listening.